Experiment 1: Install VMware Workstation with Different Flavors of Linux or Windows OS on Top of Windows 7/8

Aim

To install **VMware Workstation** on a **Windows 7/Windows 8** host operating system and create virtual machines to install different Linux or Windows operating systems.

1. Introduction

VMware Workstation is desktop virtualization software that allows multiple operating systems to run simultaneously on a single physical computer.

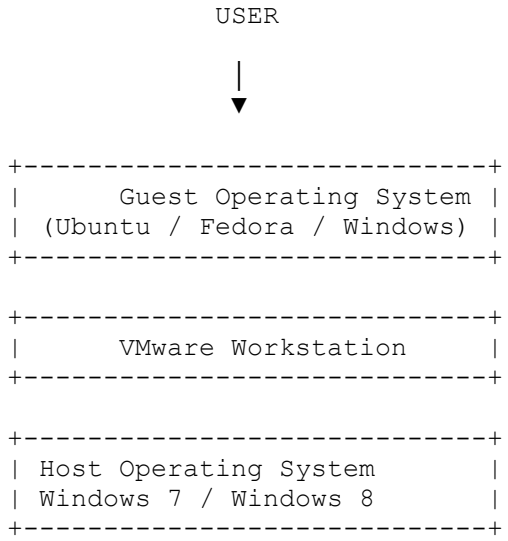
Instead of installing another operating system directly on your computer (dual boot), VMware creates a **Virtual Machine (VM)** that behaves like a real computer.

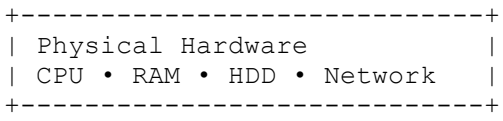
Each virtual machine has its own:

- CPU
- RAM
- Hard Disk
- Network Adapter
- Operating System

The host operating system remains unchanged while multiple guest operating systems run independently.

2. Virtualization Architecture





3. System Requirements

Component	Requirement
Processor	Intel Core i3/i5/i7 or AMD Processor
Virtualization	Intel VT-x / AMD-V Enabled
RAM	Minimum 4 GB (8 GB Recommended)
Hard Disk	40 GB Free Space
Host OS	Windows 7 or Windows 8
Software	VMware Workstation
Guest OS	Ubuntu, Fedora, Debian, Windows 10 etc.
ISO File	Bootable OS Image

4. Software Required

- VMware Workstation
- Ubuntu ISO
- Fedora ISO
- Debian ISO
- Windows ISO

Example Linux Distributions (Flavors)

- Ubuntu
- Fedora
- Debian
- Kali Linux
- Linux Mint
- CentOS

Example Windows Versions

- Windows 7
- Windows 8
- Windows 10

5. VMware Installation Process

```
Download VMware
        |
        ▼
Run Setup File
        |
        ▼
Accept License Agreement
        |
        ▼
Install VMware
        |
        ▼
Restart Computer
        |
        ▼
Launch VMware Workstation
```

6. Installing VMware Workstation

Step 1

Download VMware Workstation.

Step 2

Double-click the installer.

```
VMware-workstation.exe
```

Step 3

Click

```
Next
```

Step 4

Accept License Agreement.

```
✓ I Accept
```

Step 5

Choose Installation Folder.

Example

```
C:\Program Files\VMware\
```

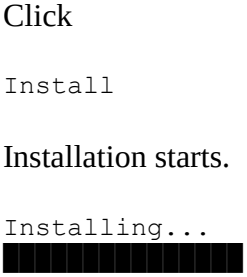
Step 6

Click

```
Install
```

Installation starts.

```
Installing...
```



Step 7

Click

```
Finish
```

Restart the computer.

7. Creating a Virtual Machine

Open VMware.

Click

```
Create New Virtual Machine
```

Diagram

```
VMware Home

+--------------------------------+
| Create a New Virtual Machine   |
+--------------------------------+
```

Choose

```
Typical Installation
(•) Typical

( ) Custom
```

Click

```
Next
```

8. Select Installation Source

Choose

```
Installer Disc Image (ISO)
```

Browse

```
ubuntu-24.iso
```

Diagram

```
Select ISO

+----------------------------+
| ubuntu.iso                 |
+----------------------------+

Browse...
```

9. Select Guest Operating System

Example

```
Linux

Ubuntu 64-bit
```

or

```
Microsoft Windows

Windows 10
```

10. Name the Virtual Machine

Example

```
Ubuntu VM
```

Location

```
D:\Virtual Machines\
```

11. Specify Disk Capacity

Example

```
40 GB
```

Diagram

```
Disk Size

40 GB

(•) Store as Single File

( ) Split into Multiple Files
```

12. Allocate RAM

Recommended

```
4096 MB
```

Diagram

```
RAM

1024 MB

2048 MB

4096 MB  ← Recommended

8192 MB
```

13. Allocate Processor

Example

```
Processors = 2

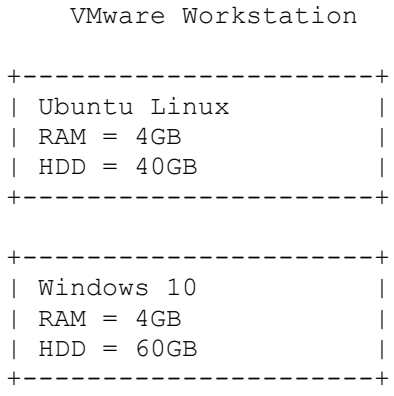
Cores = 2
```

Diagram

```
CPU

Processors : 2
Cores : 2
```

14. Virtual Machine Architecture




```
Host

Windows 8

↓

Physical Hardware
```

15. Installing Ubuntu Linux

Power ON the VM.

Ubuntu installer appears.

```
Ubuntu Installer

Install Ubuntu

Try Ubuntu
```

Click

```
Install Ubuntu
```

Select

```
English
```

Choose Keyboard Layout.

```
English (US)
```

Click

```
Continue
```

Select Installation Type

```
Erase Disk
```

```
(inside Virtual Machine only)
```

Click

```
Install Now
```

Create User

Example

```
Username : Student
```

```
Password : student123
```

Click

```
Continue
```

Ubuntu installation begins.

```
Installing Ubuntu...
```



Restart VM.

Ubuntu Desktop appears.

16. Installing Windows Guest OS

Power ON the Windows VM.

Windows Setup starts.

```
Windows Setup

Install Now
```

Choose

- Language
- Keyboard
- Time Zone

Enter Product Key (if required).

Choose

```
Custom Installation
```

Select Virtual Disk.

Click

```
Next
```

Installation starts.

```
Copying Windows Files
```

```
████████████
```

Restart.

Windows Desktop appears.

17. Install VMware Tools

VM Menu

```
VM

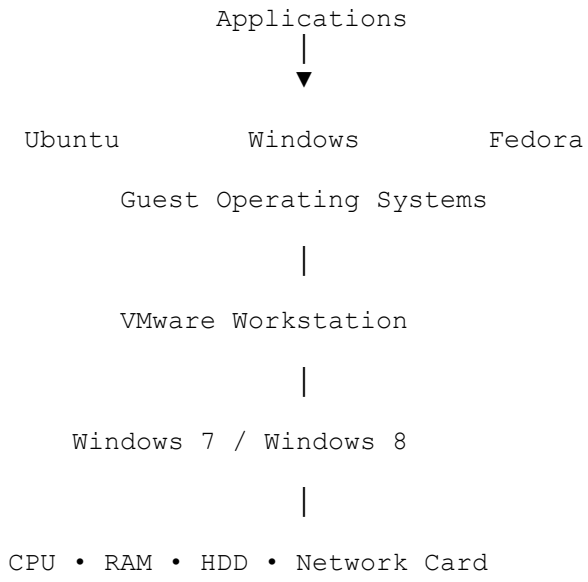
↓
```

```
Install VMware Tools
```

Benefits

- Better Graphics
- Shared Clipboard
- Drag and Drop
- Better Mouse Integration
- Better Performance
- Folder Sharing

18. Complete Virtualization Diagram



19. Advantages of VMware Workstation

- Run multiple operating systems simultaneously.
- No need to dual boot.
- Safe environment for testing software.
- Easy snapshot and restore.
- Supports networking between virtual machines.
- Portable virtual machines.
- Supports Linux and Windows guest operating systems.
- Useful for software development, testing, and education.

20. Applications

- Operating System Labs
- Network Simulation
- Cybersecurity Practice
- Cloud Computing
- Linux Administration
- Software Testing
- Application Development
- Data Science Environment Setup
- Virtual Server Deployment

21. Result

VMware Workstation was successfully installed on a Windows 7/Windows 8 host operating system. Virtual machines were created, and different guest operating systems such as Ubuntu Linux and Windows were successfully installed and executed.

Experiment 2: Install a C Compiler in a VMware Virtual Machine and Execute Simple C Programs

Aim

To install a C compiler (GCC) in a VMware virtual machine and compile and execute simple C programs.

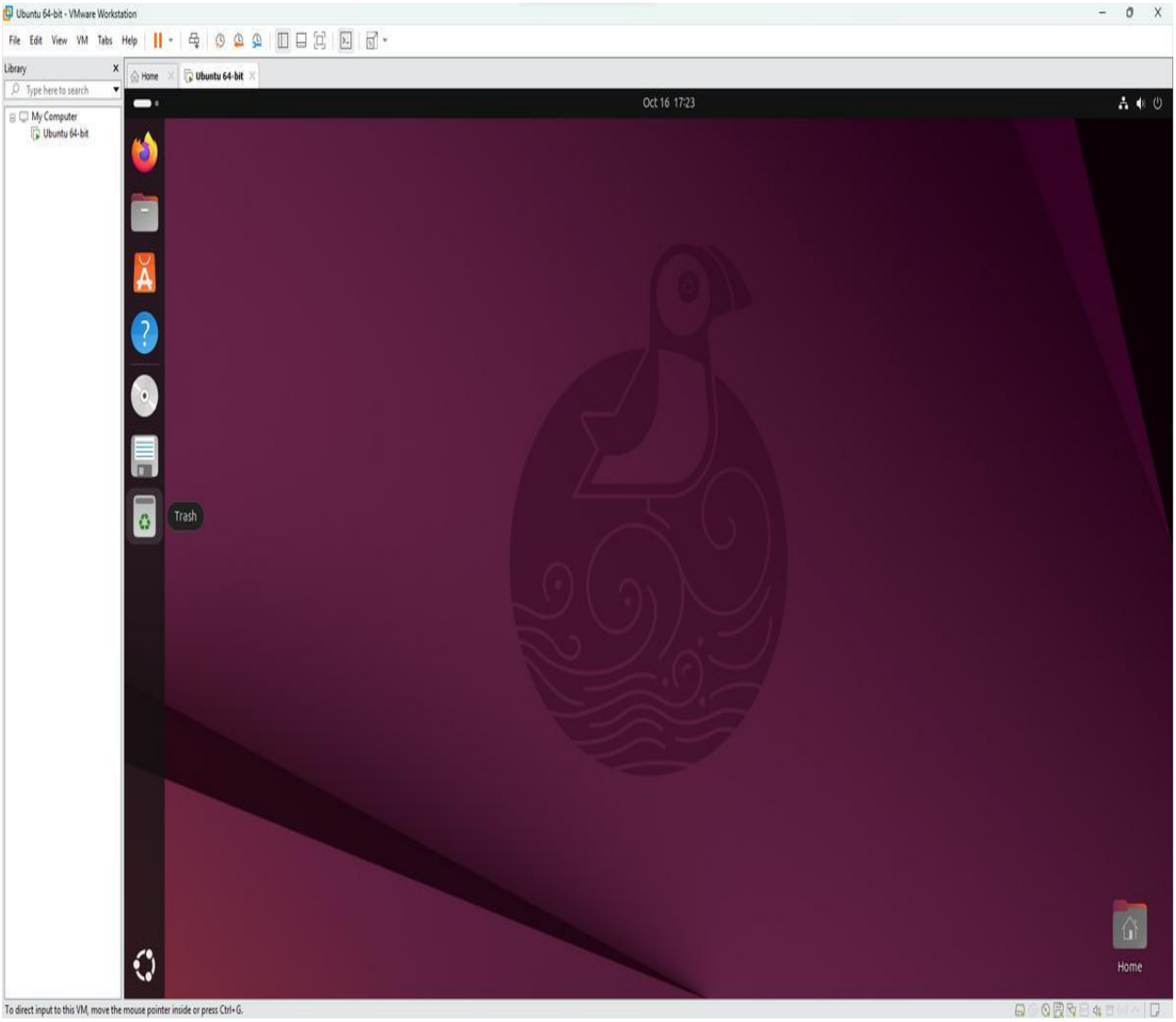
Requirements

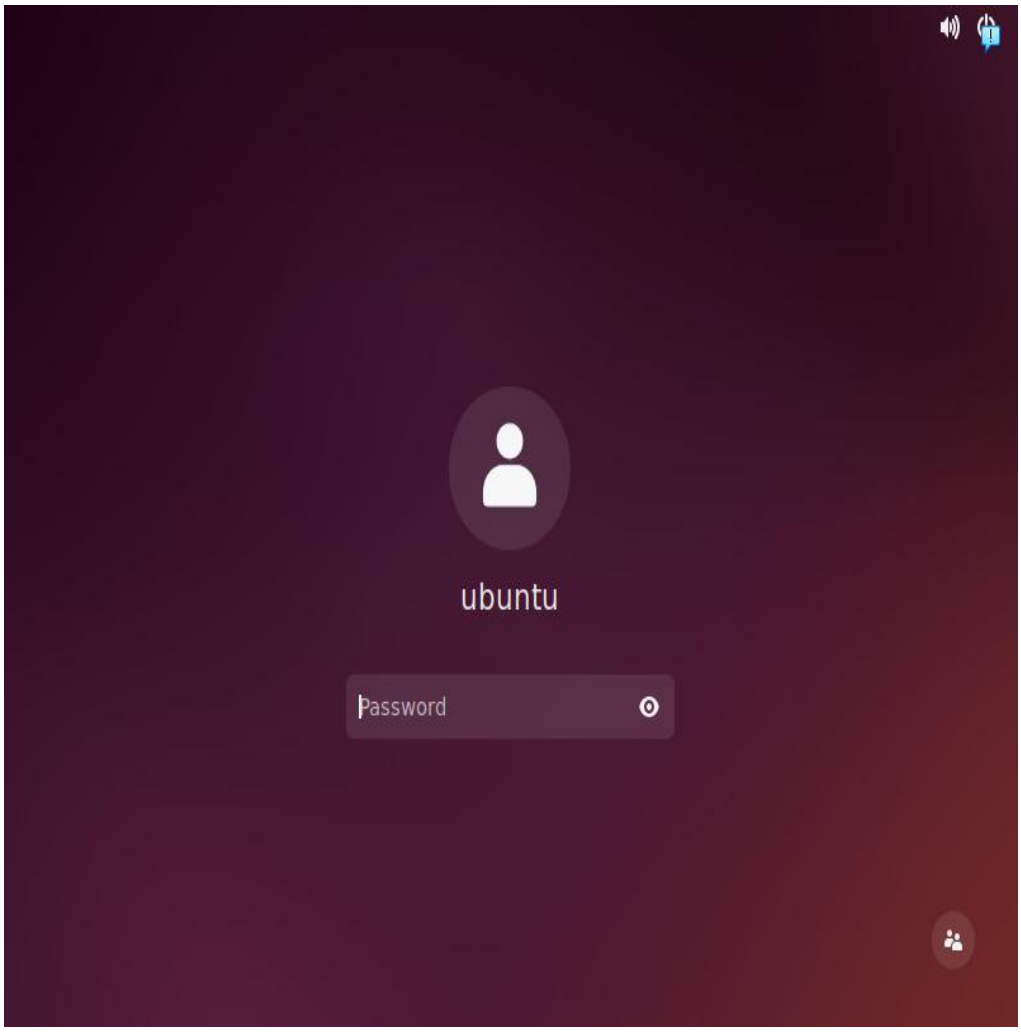
- VMware Workstation (Player/Pro)
- Ubuntu Linux Virtual Machine (or any Linux distribution)
- Internet connection
- GCC (GNU Compiler Collection)

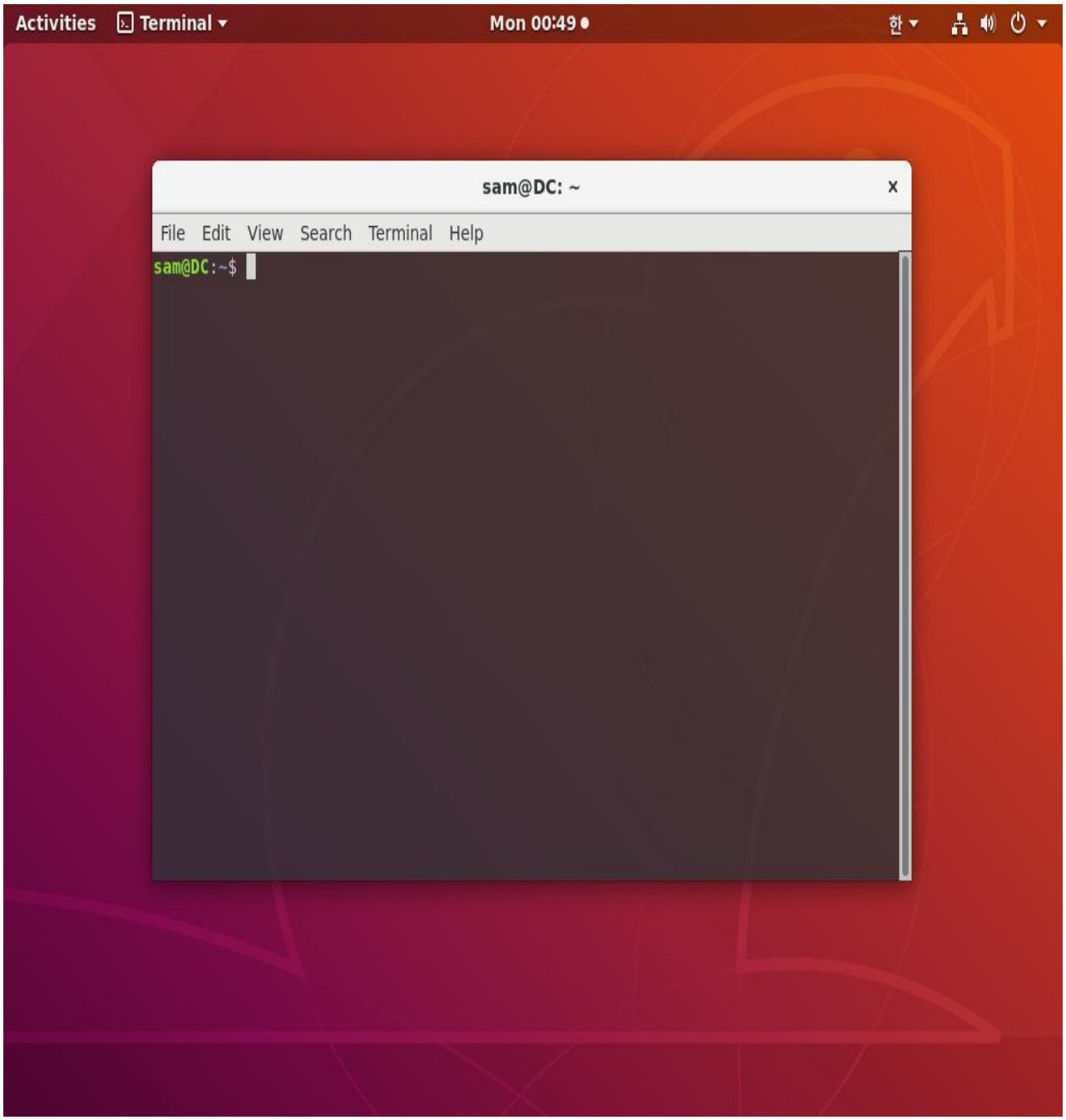
Procedure

Step 1: Start VMware and Launch the Virtual Machine

1. Open **VMware Workstation**.
2. Select the Ubuntu virtual machine.
3. Click **Power On This Virtual Machine**.
4. Log in with your username and password.




ubuntu
Password



6

Step 2: Open the Terminal

Open the terminal using either of the following methods:

- Press **Ctrl + Alt + T**
- Click **Applications → Terminal**

Step 3: Update the Package Repository

Run the following command:

```
sudo apt update
```

Enter the administrator password if prompted.

Example output:

```
Hit:1 http://archive.ubuntu.com/ubuntu
Reading package lists... Done
Building dependency tree... Done
```

Step 4: Install the GCC C Compiler

Install GCC using the following command:

```
sudo apt install gcc
```

When prompted:

```
Do you want to continue? [Y/n]
```

Press **Y** and then **Enter**.

The installation will complete in a few minutes.

Step 5: Verify the Installation

Check whether GCC has been installed successfully:

```
gcc --version
```

Example output:

```
gcc (Ubuntu) 13.x.x
Copyright (C) Free Software Foundation
```

This confirms that the compiler has been installed successfully.

Executing Simple C Programs

Program 1: Hello World

Create the Program

```
nano hello.c
```

Type the following code:

```
#include <stdio.h>

int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

Save the file:

- Ctrl + O
- Enter
- Ctrl + X

Compile the Program

```
gcc hello.c -o hello
```

Execute the Program

```
./hello
```

Output

```
Hello, World!
```

Program 2: Addition of Two Numbers

Create the file:

```
nano addition.c
```

Program:

```
#include <stdio.h>

int main()
{
```

```
    int a = 10, b = 20;

    printf("Sum = %d\n", a + b);

    return 0;
}
```

Compile:

```
gcc addition.c -o addition
```

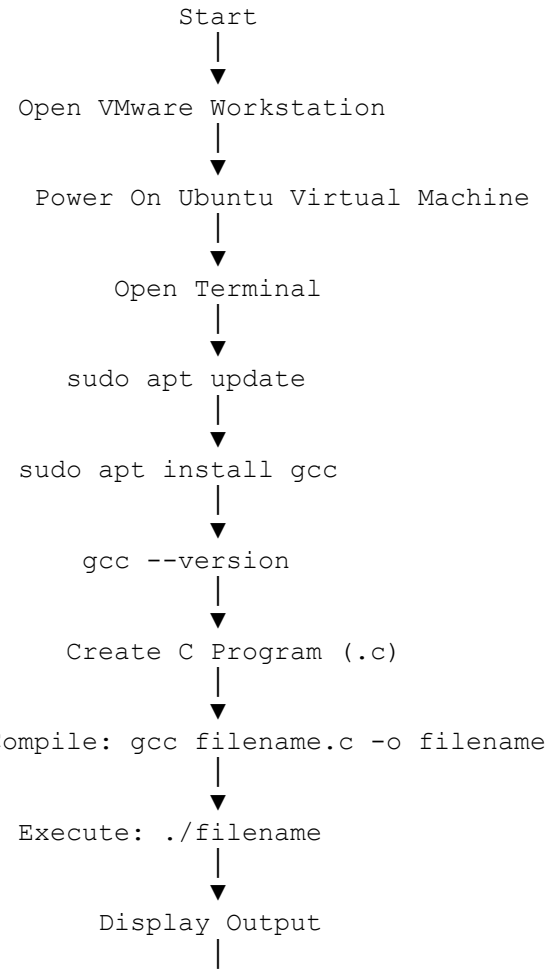
Execute:

```
./addition
```

Output:

```
Sum = 30
```

Flow Diagram





Advantages of GCC

- Free and open-source C compiler.
- Fast and reliable compilation.
- Supports C, C++, Objective-C, and other languages.
- Available on almost all Linux distributions.
- Ideal for learning C programming and software development.

Applications

- Operating system development
- Embedded systems programming
- System software development
- Academic laboratory programming
- Data structures and algorithm implementation
- General software development

Conclusion

The GCC C compiler was successfully installed in the VMware Ubuntu virtual machine. The installation was verified using the `gcc --version` command. Several simple C programs were created, compiled, and executed successfully, demonstrating the complete workflow of C program development in a VMware-based Linux environment.

Experiment 3: Create an Amazon EC2 Instance, Set Up a Web Server, and Associate an Elastic IP Address

Aim

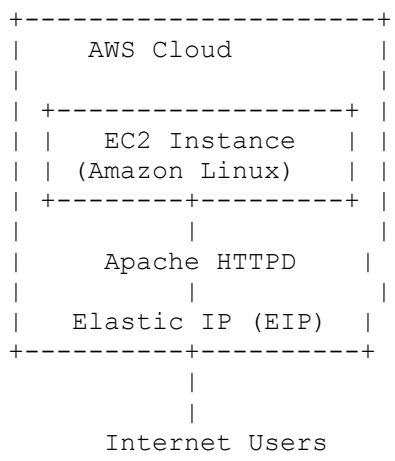
To create an Amazon EC2 instance, install and configure a web server, and associate an Elastic IP address with the instance.

Requirements

- AWS Account

- Internet connection
- Web browser
- AWS Management Console access

Architecture Diagram



Step 1: Login to AWS

1. Open your browser.
2. Go to **https://aws.amazon.com/**
3. Click **Sign In to Console**.
4. Enter your AWS credentials.

Step 2: Launch an EC2 Instance

1. In AWS Console search for **EC2**.
2. Click **EC2 Dashboard**.
3. Click **Launch Instance**.

Configure Instance

Parameter	Value
Name	WebServer
AMI	Amazon Linux 2023
Instance Type	t2.micro (Free Tier Eligible)
Key Pair	Create or Select Existing
Network	Default VPC
Auto Assign Public IP	Enabled

Security Group

Allow the following inbound rules:

Type	Port	Source
SSH	22	My IP
HTTP	80	Anywhere (0.0.0.0/0)
HTTPS	443	Anywhere

Click **Launch Instance**.

Step 3: Connect to the EC2 Instance

Select the instance.

Click **Connect**.

Choose **EC2 Instance Connect**.

Click **Connect**.

Alternatively, use SSH:

```
ssh -i mykey.pem ec2-user@Public-IP
```

Step 4: Update Packages

Run:

```
sudo yum update -y
```

For Amazon Linux 2023, you may also use:

```
sudo dnf update -y
```

Step 5: Install Apache Web Server

Install Apache:

```
sudo yum install httpd -y
```

or

```
sudo dnf install httpd -y
```

Step 6: Start the Web Server

```
sudo systemctl start httpd
```

Enable the service:

```
sudo systemctl enable httpd
```

Check status:

```
sudo systemctl status httpd
```

Expected output:

```
Active: active (running)
```

Step 7: Create a Sample Web Page

Open the HTML file:

```
sudo nano /var/www/html/index.html
```

Add:

```
<!DOCTYPE html>
<html>
<head>
<title>My AWS Web Server</title>
</head>

<body>

<h1>Welcome to AWS EC2</h1>

<h2>Web Server Successfully Configured</h2>

<p>This page is hosted on Amazon EC2.</p>

</body>
</html>
```

Save:

```
Ctrl + O
Enter
Ctrl + X
```

Step 8: Test the Website

Copy the EC2 Public IP.

Example:

```
http://54.xx.xx.xx
```

You should see:

```
Welcome to AWS EC2

Web Server Successfully Configured
```

Step 9: Allocate an Elastic IP

1. Open **EC2 Dashboard**
2. Click **Elastic IPs**
3. Click **Allocate Elastic IP Address**
4. Click **Allocate**

AWS assigns a static public IP.

Step 10: Associate Elastic IP

1. Select the Elastic IP.
2. Click **Actions**
3. Choose **Associate Elastic IP Address**
4. Select the EC2 instance.
5. Choose the private IP.
6. Click **Associate**.

Step 11: Verify

Open the browser:

```
http://Elastic-IP
```

Example:

```
http://13.xxx.xxx.xxx
```

The webpage should load successfully.

AWS Resources Used

- Amazon EC2
- Amazon Linux 2023
- Apache HTTP Server (httpd)
- Elastic IP
- Security Group
- VPC

Commands Used

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
sudo nano /var/www/html/index.html
```

Result

An Amazon EC2 instance was successfully created using Amazon Linux. The Apache web server was installed and configured, a sample web page was hosted on the instance, and an Elastic IP address was allocated and associated with the EC2 instance. The website was successfully accessed through the Elastic IP address.

4.Google App Engine Installation and Hello World Applications

Prerequisites

Before installing Google App Engine, ensure you have:

- A Google Account
- Python 3.9 or later
- Java JDK 17 or later (for Java applications)
- Git (optional)
- Internet connection

Step 1: Install Google Cloud SDK

1. Download the Google Cloud SDK from the official website: https://cloud.google.com/sdk/docs/install
2. Install the SDK.
3. Verify installation:

```
gcloud version
```

Expected output:

```
Google Cloud SDK xxx.x.x
```

Step 2: Initialize the SDK

Run:

```
gcloud init
```

Login using your Google account.

Choose:

- Project
- Region
- Default configuration

Step 3: Enable App Engine

Create App Engine:

```
gcloud app create
```

Choose a region such as:

```
us-central
asia-south1
europe-west
```

Hello World Application in Python

Step 1: Create Project Folder

```
hello-python/
```

Files:

```
hello-python
│
├── main.py
├── app.yaml
└── requirements.txt
```

---

main.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello World from Google App Engine!"

if __name__ == "__main__":
    app.run()
```

---

requirements.txt

```
Flask==2.3.2
```

```
gunicorn==21.2.0
```

---

app.yaml

```
runtime: python311

entrypoint: gunicorn -b :$PORT main:app
```

Deploy

```
gcloud app deploy
```

Open the application:

```
gcloud app browse
```

Output:

```
Hello World from Google App Engine!
```

Simple Python Web Application

main.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Welcome to My Website</h1>"

@app.route("/about")
def about():
    return "<h2>About Page</h2>"

@app.route("/contact")
def contact():
    return "<h2>Contact Page</h2>"

if __name__ == "__main__":
    app.run()
```

Pages:

```
/
/about
/contact
```

Hello World Application in Java

Project Structure

```
hello-java
│
├── src
│   └── main
│       └── java
│           └── HelloServlet.java
│
├── pom.xml
└── app.yaml
```

HelloServlet.java

```
import java.io.IOException;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest req,
                      HttpServletResponse resp)
                      throws IOException {

        resp.setContentType("text/plain");
        resp.getWriter().println("Hello World from Google App Engine!");
    }
}
```

---

app.yaml

```
runtime: java17
```

Deploy

Build the project:

```
mvn package
```

Deploy:

```
gcloud app deploy
```

Open:

```
gcloud app browse
```

Output:

```
Hello World from Google App Engine!
```

Simple Java Web Application

```
import java.io.IOException;
import javax.servlet.http.*;

public class MyServlet extends HttpServlet {

    public void doGet(HttpServletRequest req,
                      HttpServletResponse resp)
                      throws IOException {

        String path = req.getRequestURI();

        resp.setContentType("text/html");

        if(path.equals("/")) {
            resp.getWriter().println("<h1>Home Page</h1>");
        }

        else if(path.equals("/about")) {
            resp.getWriter().println("<h1>About Us</h1>");
        }

        else if(path.equals("/contact")) {
            resp.getWriter().println("<h1>Contact Us</h1>");
        }
    }
}
```

Useful Google Cloud Commands

Check configuration:

```
gcloud config list
```

List projects:

```
gcloud projects list
```

Deploy application:

```
gcloud app deploy
```

Browse application:

```
gcloud app browse
```

View logs:

```
gcloud app logs tail -s default
```

Delete old versions:

```
gcloud app versions delete VERSION_ID
```

Expected Output

After deployment, visiting your App Engine URL will display:

- **Python Hello World:** `Hello World from Google App Engine!`
- **Python Simple Web App:** Home, About, and Contact pages.
- **Java Hello World:** `Hello World from Google App Engine!`
- **Java Simple Web App:** Home, About, and Contact pages.

These examples demonstrate the basic workflow for creating, deploying, and running web applications on Google App Engine using both Python (Flask) and Java.

5. Experiment: Simulate a Cloud Scenario Using CloudSim and Implement a New Scheduling Algorithm

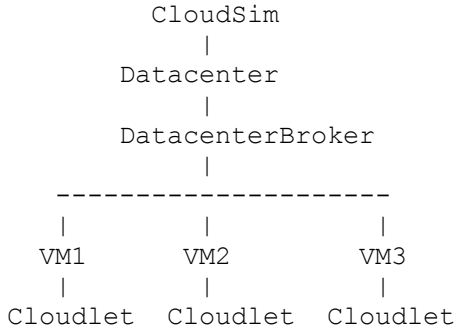
Aim

To simulate a cloud computing environment using **CloudSim** and implement a **Priority-Based scheduling algorithm** for assigning cloudlets to virtual machines.

Software Requirements

- Java JDK
- Eclipse / IntelliJ IDEA / NetBeans
- CloudSim library
- CloudSim 3.0.3 or compatible version

Basic Architecture



Scheduling Algorithm

In the normal CloudSim example, cloudlets can be assigned to VMs using simple allocation logic. We can implement **Priority-Based Scheduling** ourselves.

Each cloudlet is given a priority:

```
Priority 1 → Highest
Priority 2 → Medium
Priority 3 → Lowest
```

The scheduler sorts cloudlets according to priority and assigns the highest-priority cloudlets first.

Java Program

```
import java.util.*;

class Cloudlet {
    int id;
    int length;
    int priority;

    Cloudlet(int id, int length, int priority) {
        this.id = id;
        this.length = length;
        this.priority = priority;
    }
}

class VM {
    int id;

    VM(int id) {
```

```
        this.id = id;
    }
}

public class PriorityScheduling {

    public static void main(String[] args) {

        // Create VMs
        List<VM> vms = new ArrayList<>();

        vms.add(new VM(0));
        vms.add(new VM(1));
        vms.add(new VM(2));

        // Create Cloudlets
        List<Cloudlet> cloudlets = new ArrayList<>();

        cloudlets.add(new Cloudlet(0, 4000, 3));
        cloudlets.add(new Cloudlet(1, 2000, 1));
        cloudlets.add(new Cloudlet(2, 3000, 2));
        cloudlets.add(new Cloudlet(3, 5000, 1));
        cloudlets.add(new Cloudlet(4, 1000, 3));

        // Priority-Based Scheduling
        Collections.sort(cloudlets,
                Comparator.comparingInt(c -> c.priority));

        System.out.println("Priority Based Cloudlet Scheduling");
        System.out.println("----------------------------------");

        int vmIndex = 0;

        for (Cloudlet cloudlet : cloudlets) {

            VM vm = vms.get(vmIndex);

            System.out.println(
                "Cloudlet " + cloudlet.id +
                " | Length = " + cloudlet.length +
                " | Priority = " + cloudlet.priority +
                " | Assigned to VM " + vm.id
            );

            vmIndex = (vmIndex + 1) % vms.size();
        }
    }
}
```

Expected Output

```
Priority Based Cloudlet Scheduling
----------------------------------
Cloudlet 1 | Length = 2000 | Priority = 1 | Assigned to VM 0
Cloudlet 3 | Length = 5000 | Priority = 1 | Assigned to VM 1
Cloudlet 2 | Length = 3000 | Priority = 2 | Assigned to VM 2
```

```
Cloudlet 0 | Length = 4000 | Priority = 3 | Assigned to VM 0
Cloudlet 4 | Length = 1000 | Priority = 3 | Assigned to VM 1
```

How to Implement It in Actual CloudSim

For a complete CloudSim simulation, create:

1. **Datacenter**
2. **Host**
3. **VMs**
4. **DatacenterBroker**
5. **Cloudlets**
6. **Custom scheduling algorithm**
7. Submit VMs and cloudlets
8. Start the CloudSim simulation
9. Collect and display execution results

The important modification is the **custom scheduler**, where the default cloudlet ordering is replaced by your priority-based ordering.

Algorithm

```
START
   ↓
Initialize CloudSim
   ↓
Create Datacenter
   ↓
Create Hosts
   ↓
Create VMs
   ↓
Create Cloudlets
   ↓
Assign priority to each Cloudlet
   ↓
Sort Cloudlets by Priority
   ↓
Select VM
   ↓
Assign highest-priority Cloudlet
   ↓
Repeat until all Cloudlets are assigned
   ↓
Start CloudSim Simulation
   ↓
Display Cloudlet execution results
   ↓
STOP
```

Result

Thus, a cloud computing environment was simulated using CloudSim, and a custom Priority-Based Task Scheduling algorithm, which is not directly available as a standard scheduling policy in basic CloudSim examples, was implemented successfully.

Alternative algorithms you can implement as a CloudSim experiment:

- Shortest Job First (SJF)
- Round Robin with dynamic time quantum
- Earliest Deadline First (EDF)
- Min-Min scheduling
- Max-Min scheduling
- Priority + Load Balancing
- Energy-Aware Scheduling

If this is for your **Cloud Computing Lab record**, I can also give you the **complete CloudSim 3.0.3 Java program with Datacenter, Host, VM, Broker, Cloudlets, custom scheduling algorithm, output, algorithm, and result** in record format.

6. Procedure to Transfer Files from One Virtual Machine to Another Virtual Machine

Aim

To transfer files from one virtual machine (VM) to another VM using a network connection.

Requirements

- Oracle VirtualBox or VMware
- Two Linux virtual machines (VM1 and VM2)
- Network connectivity between the two VMs
- SSH service installed on both VMs

Procedure: Using SCP (Secure Copy Protocol)

Step 1: Configure the Network

1. Start both virtual machines.
2. Set their network adapters to the same network, such as Host-Only Adapter or NAT Network in VirtualBox.
3. Ensure both VMs can communicate with each other.

Step 2: Find the IP Address of VM2

Open the terminal in VM2 and execute:

```
ip addr
```

Note the IP address, for example, `192.168.56.102`.

Step 3: Install and Start SSH

On Ubuntu/Debian, execute the following commands on VM2:

```
sudo apt update
sudo apt install openssh-server
sudo systemctl enable --now ssh
```

Check the SSH service:

```
sudo systemctl status ssh
```

Step 4: Create a File on VM1

On VM1, create a sample file:

```
echo "Hello from VM1" > sample.txt
```

Step 5: Transfer the File from VM1 to VM2

On VM1, run:

```
scp sample.txt username@192.168.56.102:/home/username/
```

Replace `username` with the actual username on VM2 and the IP address with VM2's actual IP address. Enter VM2's password when prompted.

To transfer a directory, use:

```
scp -r myfolder username@192.168.56.102:/home/username/
```

Step 6: Verify the Transfer

On VM2, execute:

```
ls -l
cat sample.txt
```

The file should be present, and its contents should display:

```
Hello from VM1
```

Alternative Method: Using Shared Folders

In VirtualBox, you can configure a shared folder accessible to both VMs if they can access the same host folder. Enable Shared Folders in the VM settings, install Guest Additions if required, and copy files through the shared directory.

Result

Files were successfully transferred from one virtual machine to another using the SCP command over an SSH connection.

Conclusion

SCP provides a secure and reliable way to transfer files between Linux virtual machines over a network.

7. Procedure to Launch a Virtual Machine Using TryStack (Online OpenStack Demo)

Aim

To launch and access a virtual machine (VM) using TryStack, an online demonstration environment for OpenStack.

Requirements

- Internet connection
- Web browser
- TryStack account and login credentials
- Access to the OpenStack Horizon dashboard
- A suitable image, flavor, network, and security group

Procedure

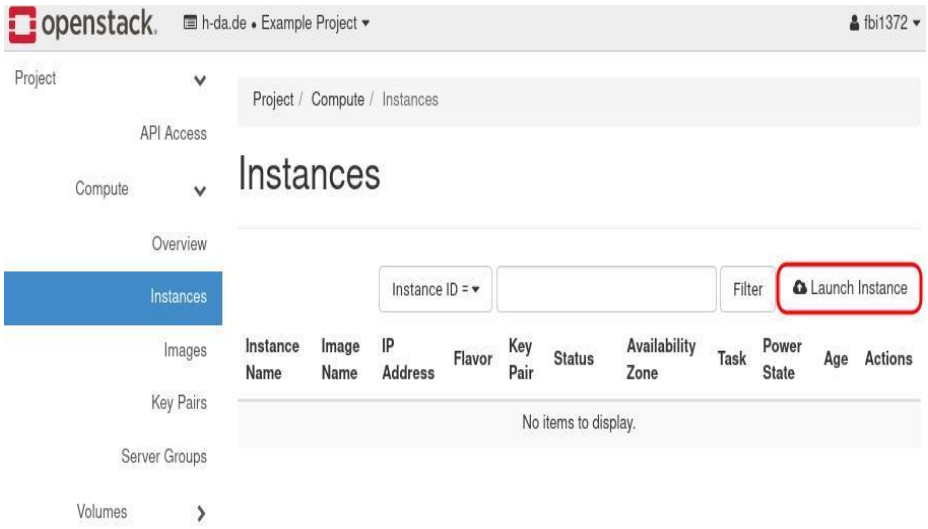
Step 1: Open TryStack

1. Open a web browser.
2. Visit the TryStack website or the demo URL provided by your faculty.
3. Log in using your registered credentials.

Note: TryStack's availability and registration process may change. If the original demo is unavailable, use an OpenStack training environment provided by your institution.

Step 2: Access the OpenStack Dashboard

1. Open the OpenStack Horizon dashboard after logging in.
2. Navigate to Project → Compute → Instances.
3. Click Launch Instance.



hdacloud.h-da.io

Step 3: Configure the Instance

Enter the following details in the launch wizard:

- Instance Name: `Test-VM`
- Availability Zone: Select the default option.
- Count: `1`

Click Next or proceed to the next configuration tab.

Step 4: Select an Image

1. Open the Source or Image tab.
2. Select an available operating system image, such as Ubuntu or CirrOS.
3. Choose the option to create a new instance from the image, if prompted.
4. Allocate the image to the instance.

Step 5: Select a Flavor

1. Open the Flavor tab.
2. Select an available flavor, such as `m1.tiny`.
3. Check that the selected flavor has sufficient memory and disk space.

Step 6: Configure Networking

1. Open the Networks tab.
2. Select an available network.
3. Ensure that the network is attached to the instance.

Step 7: Configure Security

1. Open the Security Groups tab.
2. Select the default security group or an appropriate configured group.
3. If SSH access is required, ensure that TCP port 22 is permitted and that a key pair is configured.

Step 8: Launch the Virtual Machine

1. Review all the configuration details.
2. Click Launch Instance.
3. Wait until the instance status changes from `Build` to `Active`.

Step 9: Access the Virtual Machine

1. Select the instance name from the Instances page.
2. Check its status and assigned IP address.
3. Click Console to access the VM through the browser.
4. If SSH access is configured, connect using the private key:

```
ssh -i MyKey.pem ubuntu@<floating-ip>
```

Replace `<floating-ip>` with the actual IP address assigned to your instance.



OpenStack Docs

+1

Expected Output

Instances

Instance Name | Filter | Filter | Launch Instance | Terminate Instances | More Actions

Instance Name	Image Name	IP Address	Size	Key Pair	Status	Availability Zone	Task	Power State	Time since created	Actions
ocs01	centos7-image	192.168.101.6 Floating IPs: 192.168.1.30	nuage.osc	pin-laptop	Active	nova	None	Running	1 hour, 55 minutes	Create Snapshot
vsc01	VSC32R6	192.168.101.5	nuage.vsc	pin-laptop	Active	nova	None	Running	5 days, 7 hours	Create Snapshot
vsd01	snap-0509-vsd32r6	192.168.101.4 Floating IPs: 192.168.1.29	nuage.vsd	pin-laptop	Active	nova	None	Running	5 days, 21 hours	Create Snapshot
jumpbox	snap-0509-jumpbox	192.168.101.3 Floating IPs: 192.168.1.28	nuage.tiny	pin-laptop	Active	nova	None	Running	5 days, 22 hours	Create Snapshot

Displaying 4 items

The Instances page should display the created VM with its name, image, flavor, IP address, and status `Active`.

Result

The procedure for launching a virtual machine using the OpenStack Horizon dashboard was studied. The VM can be accessed through the web console or SSH when the required access settings are configured.

8. Install Hadoop Single-Node Cluster and Run Word Count Application

Aim

To install and configure Apache Hadoop in pseudo-distributed (single-node cluster) mode and execute a simple MapReduce application, such as Word Count.

Requirements

- Ubuntu Linux (VirtualBox VM or physical machine)
- Java JDK
- Apache Hadoop
- Terminal access
- Internet connection

Procedure

Step 1: Install Java

Open the terminal and execute:

```
sudo apt update
sudo apt install openjdk-11-jdk ssh rsync -y
```

Check the Java installation:

```
java -version
javac -version
```

Step 2: Download Hadoop

Download a compatible Hadoop release from the official Apache website. For this example, use Hadoop 3.3.6.

```
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R $USER:$USER /usr/local/hadoop
```

Step 3: Configure environment variables

Open the Bash configuration file:

```
nano ~/.bashrc
```

Add these lines at the end:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Save the file, then run:

```
source ~/.bashrc
hadoop version
```

If `JAVA_HOME` differs on your system, find the correct path using:

```
readlink -f $(which java)
```

Step 4: Configure SSH passwordless login

Hadoop uses SSH to start and manage its services.

```
sudo service ssh start
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost
```

If prompted to trust the host, type `yes`. After testing, type `exit` to return to your terminal.

Step 5: Configure Hadoop files

Set the Java path in Hadoop's environment configuration:

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

Add or update:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

Now configure the following XML files in `$HADOOP_HOME/etc/hadoop/`.

1. `core-site.xml`

```
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

2. `hdfs-site.xml`

```
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>file:///home/USER/hadoopdata/namenode</value>
    </property>
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:///home/USER/hadoopdata/datanode</value>
    </property>
</configuration>
```

Replace `USER` with your Linux username. You can find it using `whoami`.

Create the storage directories:

```
mkdir -p ~/hadoopdata/namenode
mkdir -p ~/hadoopdata/datanode
```

3. `mapred-site.xml`

```
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
```

4. `yarn-site.xml`

```
<configuration>
    <property>
```

```
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
</configuration>
```

To edit these files, use commands such as:

```
nano $HADOOP_HOME/etc/hadoop/core-site.xml
nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

For `mapred-site.xml`, if the file does not exist, copy the template first:

```
cp $HADOOP_HOME/etc/hadoop/mapred-site.xml.template \
   $HADOOP_HOME/etc/hadoop/mapred-site.xml
```

Step 6: Format the NameNode

Run this command only when initializing a new Hadoop setup:

```
hdfs namenode -format
```

Successful formatting initializes the HDFS metadata for the cluster.

Step 7: Start Hadoop services

Start HDFS:

```
start-dfs.sh
```

Start YARN:

```
start-yarn.sh
```

Verify the running Java processes:

```
jps
```

Expected processes include:

- NameNode
- DataNode
- SecondaryNameNode
- ResourceManager
- NodeManager
- Jps

The process list can vary if a service fails to start.

Step 8: Run the Word Count application

Word Count is a MapReduce program that counts how many times each word appears in a text file.

1. Create an input file

```
mkdir -p ~/wordcount
cd ~/wordcount

echo "hadoop is useful" > input.txt
echo "hadoop is powerful" >> input.txt
echo "word count using hadoop" >> input.txt
```

2. Create an input directory in HDFS

```
hdfs dfs -mkdir -p /user/$USER/input
```

3. Upload the input file to HDFS

```
hdfs dfs -put input.txt /user/$USER/input/
```

Verify the uploaded file:

```
hdfs dfs -ls /user/$USER/input
```

4. Execute the Word Count program

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-
3.3.6.jar \
WordCount /user/$USER/input /user/$USER/output
```

Wait until the job completes successfully.

5. Display the output

```
hdfs dfs -cat /user/$USER/output/part-r-00000
```

Expected output (word order may differ):

```
count  1
hadoop  3
is  2
powerful  1
useful  1
using  1
word  1
```

The output shows the frequency of each word in the input file.

Step 9: Verify the cluster

Open these addresses in your browser from the Hadoop machine:

HDFS NameNode Web UI

View HDFS storage and DataNode status.

http://localhost:9870

YARN ResourceManager Web UI

View submitted MapReduce jobs and cluster resources.

http://localhost:8088

Result

Apache Hadoop was installed and configured in single-node cluster mode. HDFS and YARN services were started, and the Word Count MapReduce application was executed successfully to calculate the frequency of words in a text file.

9. Create a Database Instance in the Cloud Using Amazon RDS

Aim

To create a database instance in the cloud using Amazon Relational Database Service (Amazon RDS) and connect to the database.

Software Requirements

- Amazon Web Services (AWS) account
- Web browser
- Amazon RDS
- MySQL database engine (or another supported engine)

Procedure

Step 1: Sign in to AWS

1. Open the AWS Management Console: https://console.aws.amazon.com/
2. Sign in using your AWS account credentials.
3. Search for RDS in the AWS console search bar.
4. Select Amazon RDS.

Step 2: Create a Database

1. Select Databases from the navigation panel.
2. Click Create database.
3. Choose Standard create.
4. Select MySQL as the database engine.
5. Choose a suitable engine version.

Step 3: Configure Database Settings

1. Select a template such as Free tier, if available for your account.
2. Enter the DB instance identifier, for example, `mydb-instance`.
3. Enter a master username, such as `admin`.
4. Set a strong master password and confirm it.
5. Choose an instance class and storage size according to the lab requirements.

Step 4: Configure Connectivity

1. Select the appropriate Virtual Private Cloud (VPC).
2. Configure the DB subnet group.
3. Choose whether to enable public access.
4. Select or create a security group.
5. Allow inbound MySQL traffic on port 3306 only from your authorized client IP address or application security group.

For a lab, prefer private access when possible. Do not expose the database to all IP addresses.

Step 5: Create the Instance

1. Review the database configuration.
2. Click Create database.
3. Wait until the database status changes to Available.
4. Select the database instance to view its details.

Step 6: Obtain the Endpoint

1. Open the database instance details.
2. Find the Connectivity & security section.
3. Copy the database endpoint and port number.

4. The default MySQL port is `3306`.

Step 7: Connect to the Database

If MySQL client software is installed and network access is configured, execute:

```
mysql -h <RDS-ENDPOINT> -P 3306 -u admin -p
```

Replace `<RDS-ENDPOINT>` with the endpoint displayed in the AWS console. Enter the master password when prompted.

Step 8: Create a Database and Table

After connecting, execute:

```
CREATE DATABASE college;
USE college;

CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(30)
);

INSERT INTO students
VALUES (1, 'Ravi', 'CSE');

SELECT * FROM students;
```

Expected output:

```
+----+------+------------+
| id | name | department |
+----+------+------------+
|  1 | Ravi | CSE        |
+----+------+------------+
```

Result

A cloud database instance was created successfully using Amazon RDS. A sample database and table were created, and the inserted student record was retrieved.

Experiment 10: Create a Database Instance in the Cloud Using Google Cloud SQL

Aim

To create a database instance in the cloud using Google Cloud SQL and connect to the database.

Software Requirements

- Google account
- Internet connection
- Web browser
- Google Cloud Console
- Google Cloud SQL

Procedure

Step 1: Open Google Cloud Console

1. Open the browser and visit the Google Cloud Console.
2. Sign in using your Google account.
3. Create a new project or select an existing project.
4. Enable billing for the project if required.

Step 2: Enable the Cloud SQL API

1. Navigate to APIs & Services → Library.
2. Search for Cloud SQL Admin API.
3. Select it and click Enable.

Step 3: Create a Cloud SQL Instance

Cloud SQL

Create a MySQL instance

- Get Started
- Instances
- Backups
- Release Notes

Edition preset: Sandbox

Compare edition presets

Instance info

Database version *: MySQL 8.0

Minor version: MySQL 8.0.41 (default)

Enable automatic minor version upgrade Learn more

Hide minor versions

Instance ID *: td-demo

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password Generate

Set a password for the root user. Learn more

No password

Without a root password, this instance will allow anyone to connect with full administrative privileges. Set a root password to ensure only authorized users have these privileges.

Choose region and zonal availability

Summary

Cloud SQL Edition	Enterprise
Region	us-central1 (Iowa)
DB Version	MySQL 8.0
Machine type	db-custom-2-8192
vCPUs	2 vCPU
RAM	8 GB
Data Cache	Disabled
Storage	10 GB SSD
Connections	Public IP
Backup	Automated
Availability	Single zone
Point-in-time recovery	Enabled
Network throughput (MB/s)	500 of 500
IOPS	Read: 6,300 of 15,000 Write: 6,300 of 15,000
Disk throughput (MB/s)	Read: 4.8 of 240.0 Write: 4.8 of 240.0

Pricing estimate (without discounts)

These items represent Cloud SQL compute, memory and storage resources only, and reflect how you configured your instance so far. Discounts not included in estimate. Learn more

1. Open the Cloud SQL page: Cloud SQL Console.
2. Click Create instance.
3. Select a database engine, such as MySQL.
4. Enter the required details:
    - Instance ID: `student-db`
    - Root password: Create a strong password.
    - Database version: Select an available MySQL version.
5. Choose a region and an appropriate machine configuration.
6. Configure storage and connectivity settings.
7. Click Create instance and wait for the instance to become ready.

Note: Google Cloud SQL is a paid service in many configurations. Review the estimated cost before creating the instance.

Step 4: Create a Database

1. Open the newly created `student-db` instance.
2. Navigate to the Databases section.
3. Click Create database.
4. Enter the database name as `college_db`.
5. Click Create.

Step 5: Connect to the Database

1. Open the instance in Cloud SQL Console.
2. Go to Cloud SQL Studio.
3. Select the database `college_db`.
4. Authenticate using the configured database credentials.
5. Enter the following SQL commands and execute them.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(30)
);

INSERT INTO Students
VALUES (101, 'Ravi', 'CSE');

SELECT * FROM Students;
```

Step 6: Verify the Output

Expected output:

student_id	name	department
101	Ravi	CSE

The output confirms that the table was created and the record was inserted successfully.

Result

A database instance was created successfully using Google Cloud SQL. A database and table were created, sample data was inserted, and the data was retrieved using an SQL query.